Gestión de Proyectos de Inteligencia Artificial
Actividad 4.



Gibran Martínez Figueroa

11/10/2026

Actividad 4:

Descripción:

El estudiante analiza, selecciona e implementa modelos pre entrenados de Hugging Face para un caso de uso definido, utilizando Google Colab con GPU como entorno de prototipado rápido. Durante la actividad, se identifican los componentes clave del ecosistema Hugging Face (Models, Datasets y Spaces), se establecen criterios técnicos y prácticos de selección y se implementan al menos tres modelos distintos, los cuales se comparan mediante métricas estándar, como accuracy, precision, recall y F1-score.

Asimismo, se evalúa el equilibrio entre precisión y eficiencia, se analiza el desempeño en relación con el contexto del problema y se documentan de forma clara los resultados, las decisiones técnicas y las condiciones de ejecución.

Finalmente, se justifica la selección del modelo más adecuado y se organiza un repositorio en GitHub que garantice la reproducibilidad, validación y mantenimiento del prototipo.

Objetivo:

Identificar e implementar el modelo pre entrenado más adecuado para un caso de uso específico.

1. Cuaderno Colab funcional.

Link de google colab:

GPIA_A4.ipynb

Configuración de GPU

```
# ============================================================
# CELL 1 - GPU CHECK
# ============================================================

import torch

print("PyTorch:", torch.__version__)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("CUDA:", torch.version.cuda)
else:
    print("GPU no disponible")
```

```
PyTorch: 2.11.0+cu130
GPU: Tesla T4
CUDA: 13.0
```

Instalación de dependencias.

[3] ✓ 19 s

```
# ============================================================
# CELL 2 - Instalación de dependencias
# ============================================================

!pip -q install transformers
!pip -q install sentence-transformers
!pip -q install datasets
!pip -q install evaluate
!pip -q install accelerate
!pip -q install huggingface_hub
!pip -q install scikit-learn
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 4.2 MB/s eta 0:00:00
```

[4]

```
# ============================================================
# CELL 2.1 - Verificacion de instalacion
# ============================================================

import torch
import transformers
import datasets
import sentence_transformers
import sklearn

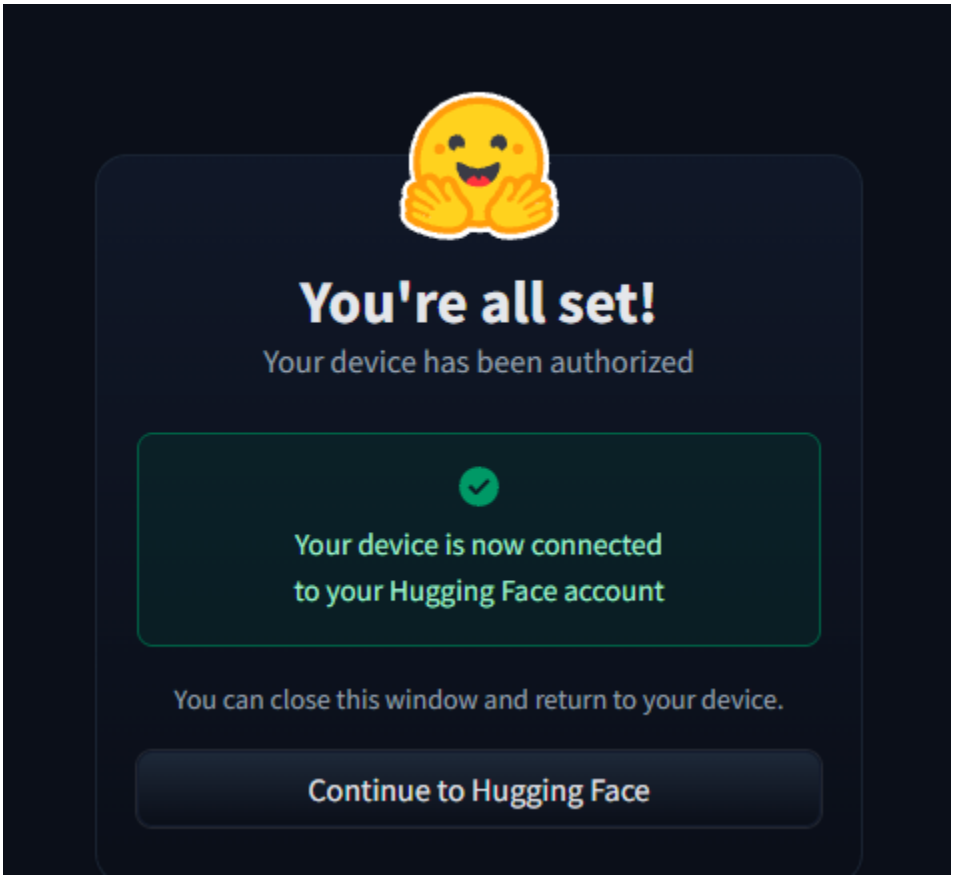
print("Torch:", torch.__version__)
print("Transformers:", transformers.__version__)
print("Datasets:", datasets.__version__)
print("Sentence Transformers:", sentence_transformers.__version__)
print("Scikit-Learn:", sklearn.__version__)

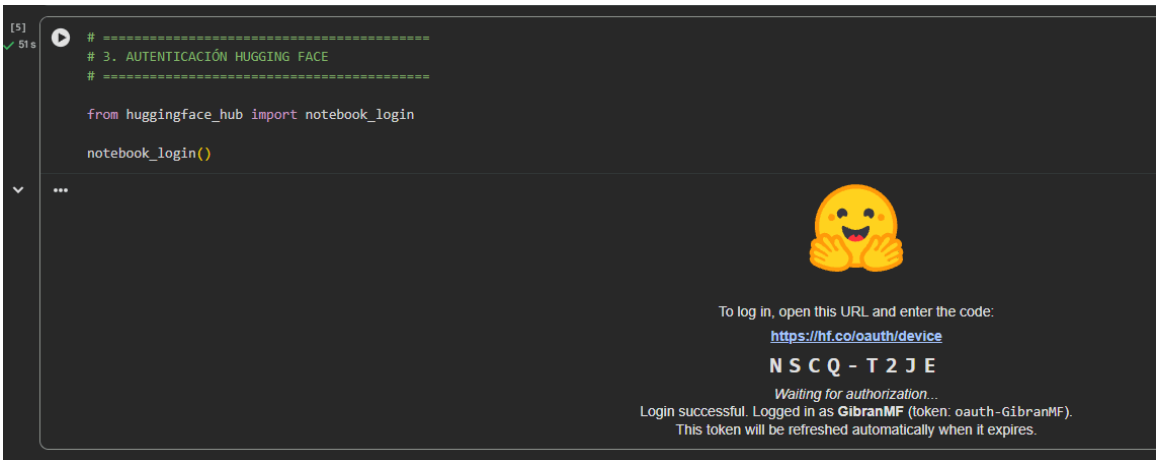
print("\nGPU Available:", torch.cuda.is_available())
```

```
Torch: 2.11.0+cu130
Transformers: 5.18.0
Datasets: 4.8.5
Sentence Transformers: 5.7.0
Scikit-Learn: 1.6.1

GPU Available: True
```

Autenticación en Hugging Face Hub.





Carga de modelos y datasets.

Justificación del Dataset y los Modelos Seleccionados

Para el desarrollo de esta actividad se seleccionó el dataset AG News, disponible a través de la plataforma Hugging Face. Este conjunto de datos contiene noticias clasificadas en cuatro categorías: World, Sports, Business y Sci/Tech. La elección de este dataset se debe a que es

ampliamente utilizado en tareas de clasificación de texto dentro del área de Procesamiento de Lenguaje Natural (NLP), además de contar con etiquetas previamente definidas que permiten realizar una evaluación objetiva de los modelos mediante métricas como Accuracy, Precision, Recall y F1-Score.

El dataset está compuesto por 120,000 registros de entrenamiento y 7,600 registros de prueba, proporcionando un volumen de datos adecuado para analizar el desempeño de distintos modelos de inteligencia artificial en una tarea de clasificación multiclase. Asimismo, al tratarse de un dataset público y ampliamente documentado, garantiza la reproducibilidad de los experimentos realizados.

Para la comparación se seleccionaron tres modelos preentrenados disponibles en Hugging Face, representando diferentes niveles de complejidad y consumo de recursos computacionales:

- facebook/bart-large-mnli: modelo basado en la arquitectura Transformer, reconocido por su alta capacidad de comprensión textual y desempeño en tareas de clasificación mediante técnicas de zero-shot learning.
- MoritzLaurer/deberta-v3-base-zeroshot-v1.1-all-33: modelo optimizado para clasificación de texto y comprensión semántica, ofreciendo un equilibrio entre precisión y eficiencia computacional.
- cross-encoder/nli-deberta-v3-base: modelo especializado en inferencia de lenguaje natural (NLI), capaz de capturar relaciones semánticas complejas entre textos.

La selección de estos modelos permite comparar distintos enfoques dentro de una misma tarea de clasificación, analizando los compromisos (trade-offs) entre precisión, velocidad de inferencia y consumo de recursos. De esta manera, es posible identificar cuál de ellos ofrece el mejor equilibrio para escenarios reales donde se requiere procesar grandes volúmenes de información textual de forma eficiente.

Caso de uso

El caso de uso elegido corresponde a la clasificación automática de noticias, una aplicación representativa del Procesamiento de Lenguaje Natural. El objetivo es que los modelos identifiquen correctamente la categoría temática de cada noticia a partir de su contenido textual, automatizando una tarea que tradicionalmente requeriría intervención humana. Este problema resulta adecuado para evaluar técnicas modernas de NLP y comparar el desempeño de diferentes modelos pre entrenados en condiciones equivalentes.

```python
# ========================================
# 5. CARGA DEL DATASET AG NEWS
# ========================================

from datasets import load_dataset

dataset = load_dataset("fancyzhx/ag_news")

print(dataset)
```

```
README.md: 100% 8.07k/8.07k [00:00<00:00, 643kB/s]
data/train-00000-of-00001.parquet: reconstructing file: 100% 18.6MB / 18.6MB, 1.83MB/s
data/train-00000-of-00001.parquet: downloading bytes: 18.5MB, 1.83MB/s
data/test-00000-of-00001.parquet: reconstructing file: 100% 1.23MB / 1.23MB, 122kB/s
data/test-00000-of-00001.parquet: downloading bytes: 1.23MB, 122kB/s
Generating train split: 100% 120000/120000 [00:00<00:00, 460403.73 examples/s]
Generating test split: 100% 7600/7600 [00:00<00:00, 201767.93 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 120000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 7600
    })
})
```

[7] ✓ 0 s

```python
# ========================================
# VISUALIZAR EJEMPLOS DEL DATASET
# ========================================

print(dataset["train"][0])
```

```
{'text': "Wall St. Bears Claw Back Into the Black (Reuters) Reuters - Short-sellers,
```

[9] ✓ 0 s

```python
# ========================================
# ETIQUETAS DEL DATASET
# ========================================

labels = dataset["train"].features["label"].names

print(labels)
```

```
['World', 'Sports', 'Business', 'Sci/Tech']
```

```
# ========================================
# TAMAÑO DEL DATASET
# ========================================

print(dataset)

print("\nTrain:", len(dataset["train"]))
print("Test:", len(dataset["test"]))
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 120000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 7600
    })
})

Train: 120000
Test: 7600
```

Se cargaron tres modelos preentrenados disponibles en Hugging Face Hub para su posterior evaluación. Los modelos seleccionados fueron Facebook BART Large MNLI, DeBERTa V3 Zero-Shot y ModernBERT Large Zero-Shot, todos ellos especializados en tareas de comprensión y clasificación de texto mediante técnicas de procesamiento de lenguaje natural. Estos modelos serán utilizados en las siguientes etapas para realizar inferencias y comparar su desempeño sobre el conjunto de datos seleccionado.

```python
# ============================================================
# MODELO 1 - FACEBOOK BART LARGE MNLI
# ============================================================
# Este modelo será utilizado para clasificar noticias mediante
# Zero-Shot Classification.
#
# Ventajas:
# - Alta precisión
# - Muy utilizado en NLP
# - Excelente comprensión semántica
#
# Desventaja:
# - Mayor consumo de memoria y mayor latencia

from transformers import pipeline

bart_classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli",
    device=0
)

print("Modelo BART cargado correctamente.")
```

```
config.json: 100% 1.15k/1.15k [00:00<00:00, 98.8kB/s]
model.safetensors: reconstructing file: 100% 1.63GB / 1.63GB, 89.9MB/s
model.safetensors: downloading bytes: 1.09GB, 85.0MB/s
Loading weights: 100% 515/515 [00:00<00:00, 3835.13it/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 2.60kB/s]
vocab.json: 100% 899k/899k [00:00<00:00, 15.4MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 10.9MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 11.6MB/s]
Modelo BART cargado correctamente.
```

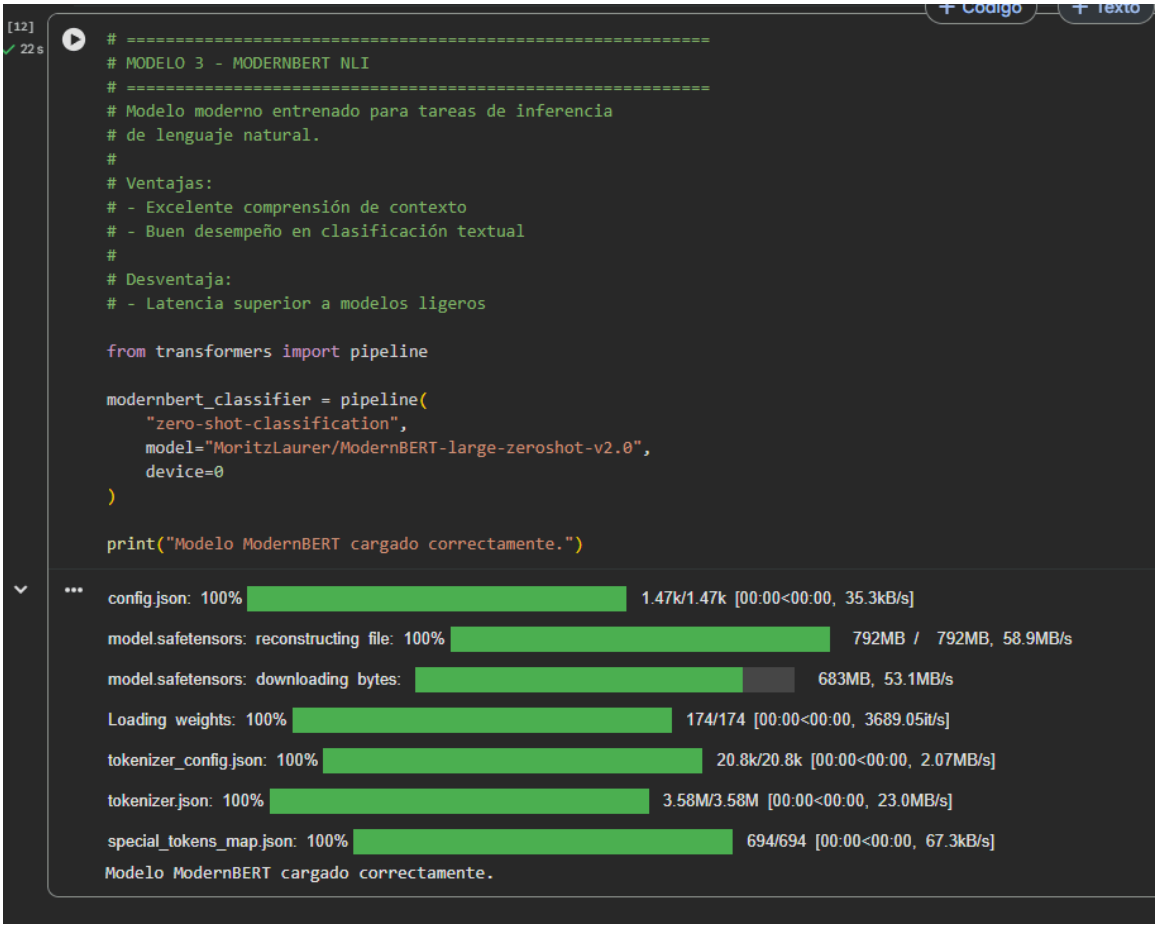
```python
# ============================================================
# MODELO 2 - DEBERTA V3 ZERO SHOT
# ============================================================
# Modelo optimizado para tareas de clasificación.
#
# Ventajas:
# - Buen equilibrio entre precisión y velocidad
# - Menor consumo de recursos
#
# Desventaja:
# - Puede ser ligeramente menos preciso que modelos más grandes

from transformers import pipeline

deberta_classifier = pipeline(
    "zero-shot-classification",
    model="MoritzLaurer/deberta-v3-base-zeroshot-v1.1-all-33",
    device=0
)

print("Modelo DeBERTa cargado correctamente.")
```

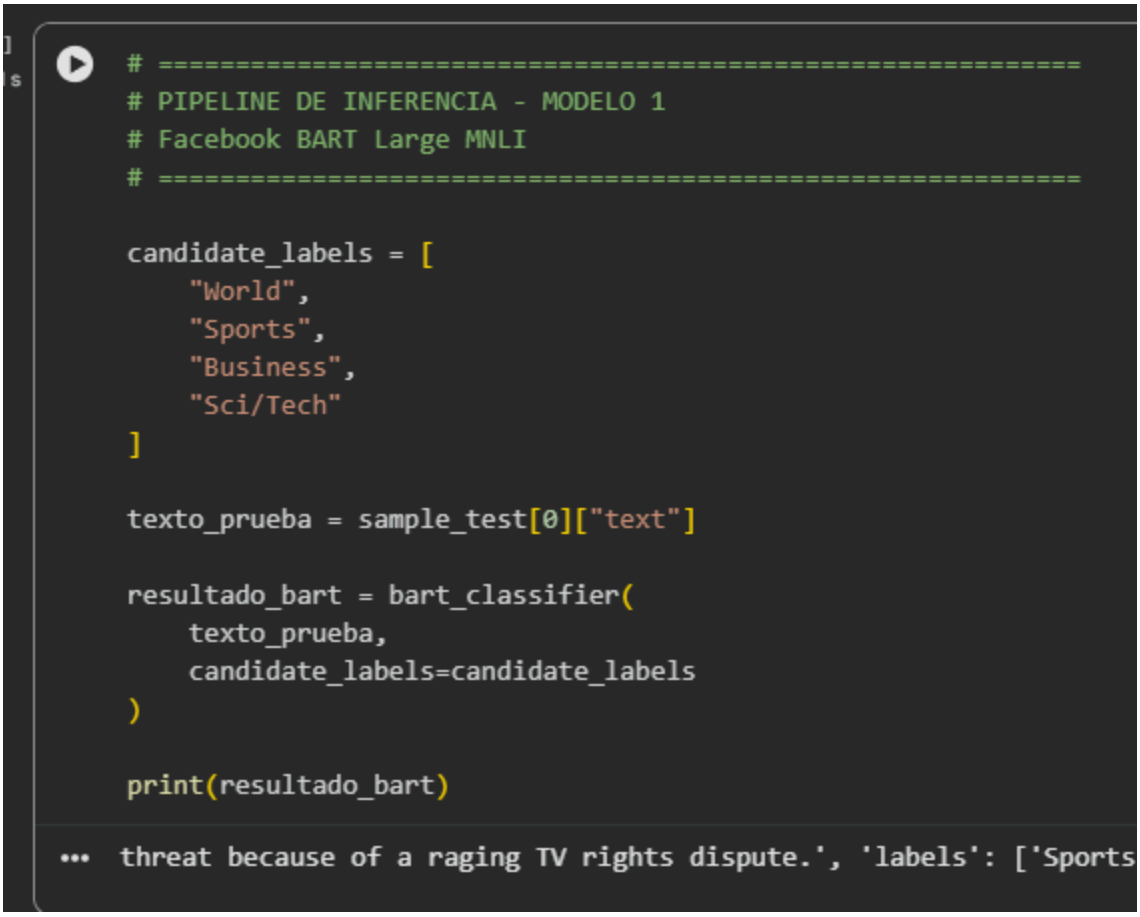
```
config.json: 100% 1.02k/1.02k [00:00<00:00, 100kB/s]
model.safetensors: reconstructing file: 100% 369MB / 369MB, 34.7MB/s
model.safetensors: downloading bytes: 369MB, 34.9MB/s
Loading weights: 100% 202/202 [00:00<00:00, 3564.32it/s]
tokenizer_config.json: 100% 1.28k/1.28k [00:00<00:00, 123kB/s]
spm.model: reconstructing file: 100% 2.46MB / 2.46MB, 244kB/s
spm.model: downloading bytes: 1.64MB, 162kB/s
tokenizer.json: 100% 8.66M/8.66M [00:00<00:00, 23.8MB/s]
added_tokens.json: 100% 23.0/23.0 [00:00<00:00, 1.60kB/s]
special_tokens_map.json: 100% 286/286 [00:00<00:00, 21.1kB/s]
Modelo DeBERTa cargado correctamente.
```



Pipeline de inferencia.

Inferencia con Facebook BART Large MNLI

Se implementó un pipeline de inferencia utilizando el modelo Facebook BART Large MNLI. Como entrada, se proporcionó una noticia perteneciente al conjunto de prueba del dataset AG News junto con las categorías candidatas definidas previamente (World, Sports, Business y Sci/Tech). El modelo generó una puntuación de confianza para cada categoría y seleccionó como predicción final aquella con la puntuación más alta. En la prueba realizada, la noticia fue clasificada como Sports, resultado consistente con el contenido deportivo del texto analizado.



```
# ============================================================
# PIPELINE DE INFERENCIA - MODELO 1
# Facebook BART Large MNLI
# ============================================================

candidate_labels = [
    "World",
    "Sports",
    "Business",
    "Sci/Tech"
]

texto_prueba = sample_test[0]["text"]

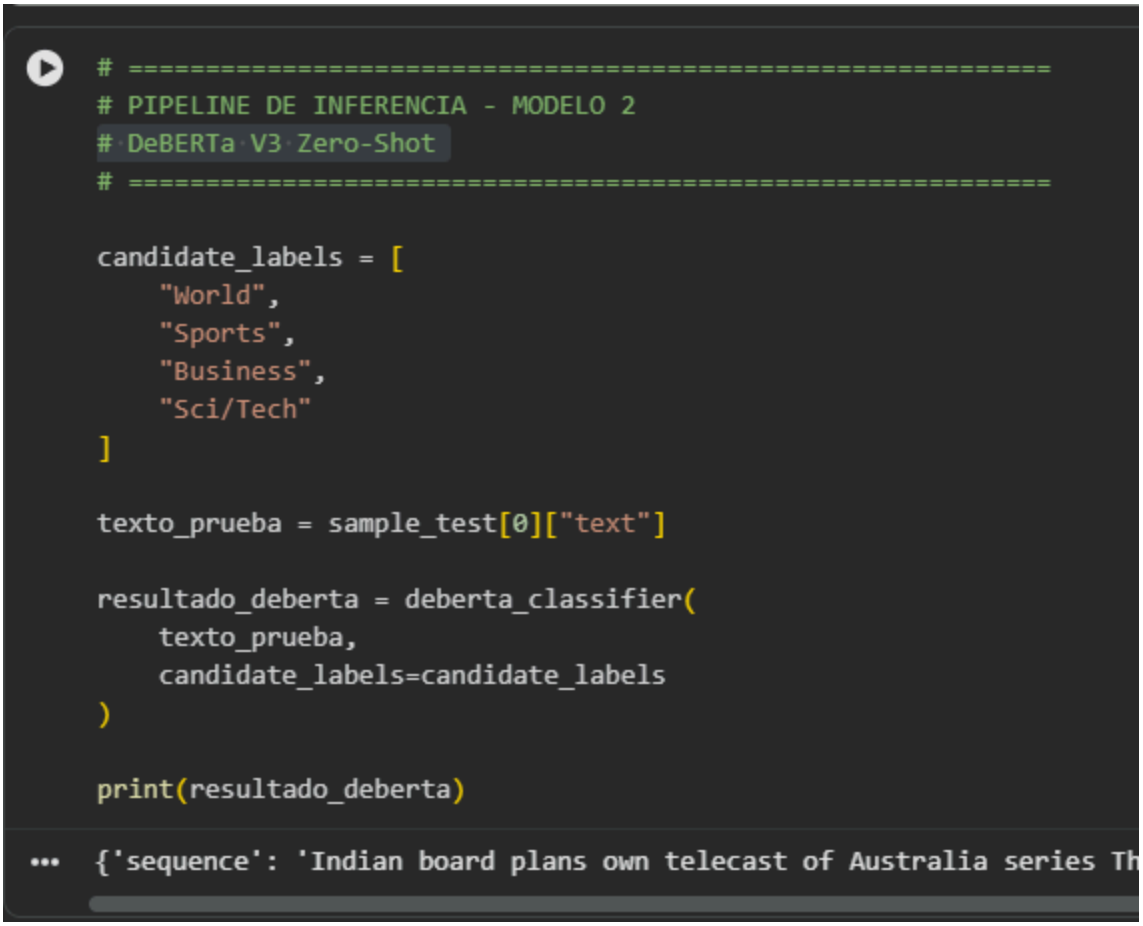
resultado_bart = bart_classifier(
    texto_prueba,
    candidate_labels=candidate_labels
)

print(resultado_bart)
```

```
... threat because of a raging TV rights dispute.', 'labels': ['Sports
```

Inferencia con DeBERTa V3 Zero-Shot

Se ejecutó el pipeline de inferencia utilizando el modelo DeBERTa V3 Zero-Shot sobre una noticia del conjunto de prueba AG News. El modelo clasificó correctamente la noticia dentro de la categoría Sports, asignando una probabilidad de confianza del 98.85%. En comparación con el modelo BART Large MNLI, DeBERTa mostró una mayor seguridad en la clasificación, evidenciando una mejor discriminación entre la categoría correcta y las categorías alternativas para este ejemplo específico.



```
# ============================================================
# PIPELINE DE INFERENCIA - MODELO 2
# DeBERTa V3 Zero-Shot
# ============================================================

candidate_labels = [
    "World",
    "Sports",
    "Business",
    "Sci/Tech"
]

texto_prueba = sample_test[0]["text"]

resultado_deberta = deberta_classifier(
    texto_prueba,
    candidate_labels=candidate_labels
)

print(resultado_deberta)
```

```
{'sequence': 'Indian board plans own telecast of Australia series Th
```

Cálculo de métricas (accuracy, precision, recall, F1-score).

Para evaluar el desempeño de los modelos se utilizaron las métricas Accuracy, Precision, Recall y F1-Score. La evaluación se realizó sobre una muestra aleatoria del conjunto de prueba del dataset AG News. Estas métricas permiten medir la capacidad de los modelos para clasificar correctamente las noticias dentro de las categorías World, Sports, Business y Sci/Tech, proporcionando una comparación objetiva entre las alternativas seleccionadas. Una vez obtenidos los resultados, se elaboró una tabla comparativa para identificar el modelo con mejor desempeño general.

```python
# ============================================================
# EVALUACIÓN - BART LARGE MNLI
# ============================================================

bart_accuracy, bart_precision, bart_recall, bart_f1 = evaluate_model(
    bart_classifier,
    sample_test
)

print("BART")
print("Accuracy :", round(bart_accuracy,4))
print("Precision:", round(bart_precision,4))
print("Recall   :", round(bart_recall,4))
print("F1 Score :", round(bart_f1,4))
```

```
[transformers] You seem to be using the pipelines sequentially on GPU. In order to
BART
Accuracy : 0.6767
Precision: 0.684
Recall   : 0.6767
F1 Score : 0.6763
```

```python
# ============================================================
# EVALUACIÓN - DEBERTA V3
# ============================================================

deberta_accuracy, deberta_precision, deberta_recall, deberta_f1 = evaluate_model(
    deberta_classifier,
    sample_test
)

print("DeBERTa")
print("Accuracy :", round(deberta_accuracy,4))
print("Precision:", round(deberta_precision,4))
print("Recall   :", round(deberta_recall,4))
print("F1 Score :", round(deberta_f1,4))
```

```
DeBERTa
Accuracy : 0.84
Precision: 0.8437
Recall   : 0.84
F1 Score : 0.8365
```

```python
# ============================================================
# EVALUACIÓN - MODERNBERT
# ============================================================

modern_accuracy, modern_precision, modern_recall, modern_f1 = evaluate_model(
    modernbert_classifier,
    sample_test
)

print("ModernBERT")
print("Accuracy :", round(modern_accuracy,4))
print("Precision:", round(modern_precision,4))
print("Recall   :", round(modern_recall,4))
print("F1 Score :", round(modern_f1,4))
```

```
ModernBERT
Accuracy : 0.8067
Precision: 0.8223
Recall   : 0.8067
F1 Score : 0.7988
```

Medición de latencia.

```python
# ============================================================
# FUNCIÓN PARA MEDIR LATENCIA
# ============================================================

import time

def measure_latency(classifier, dataset_sample):

    start_time = time.time()

    for row in dataset_sample:
        classifier(
            row["text"],
            candidate_labels=[
                "World",
                "Sports",
                "Business",
                "Sci/Tech"
            ]
        )

    end_time = time.time()

    total_time = end_time - start_time
    avg_latency = total_time / len(dataset_sample)

    return total_time, avg_latency
```

```python
# ============================================================
# LATENCIA - BART
# ============================================================

bart_total_time, bart_avg_latency = measure_latency(
    bart_classifier,
    sample_test
)

print("BART")
print("Tiempo total:", round(bart_total_time, 2), "segundos")
print("Latencia promedio:", round(bart_avg_latency, 4), "segundos")
```

```
BART
Tiempo total: 32.24 segundos
Latencia promedio: 0.1075 segundos
```

```python
# ============================================================
# LATENCIA - DEBERTA
# ============================================================

deberta_total_time, deberta_avg_latency = measure_latency(
    deberta_classifier,
    sample_test
)

print("DeBERTa")
print("Tiempo total:", round(deberta_total_time, 2), "segundos")
print("Latencia promedio:", round(deberta_avg_latency, 4), "segundos")
```

```
DeBERTa
Tiempo total: 43.15 segundos
Latencia promedio: 0.1438 segundos
```

```python
# ============================================================
# LATENCIA - MODERNBERT
# ============================================================

modern_total_time, modern_avg_latency = measure_latency(
    modernbert_classifier,
    sample_test
)

print("ModernBERT")
print("Tiempo total:", round(modern_total_time, 2), "segundos")
print("Latencia promedio:", round(modern_avg_latency, 4), "segundos")
```

```
ModernBERT
Tiempo total: 47.57 segundos
Latencia promedio: 0.1586 segundos
```

Tablas o resúmen

Resultados del Pipeline de Inferencia

Se realizó una prueba de inferencia utilizando una noticia del conjunto de prueba del dataset AG News. Los tres modelos evaluados calificaron la noticia dentro de la categoría *Sports*, coincidiendo con el contenido del texto relacionado con una serie de cricket entre India y Australia.

Los resultados mostraron diferencias en el nivel de confianza asignado por cada modelo. Facebook BART Large MNLI obtuvo una confianza del 44.30%, mientras que DeBERTa V3 Zero-Shot alcanzó un 98.85% y ModernBERT Large Zero-Shot un 94.53%. Estos resultados sugieren que, para este ejemplo específico, DeBERTa y ModernBERT presentan una mayor capacidad para distinguir la categoría correcta respecto a las alternativas disponibles.

La consistencia de las predicciones confirma que los modelos fueron cargados correctamente y que el pipeline de inferencia funciona de manera adecuada para la tarea de clasificación de noticias.

Tabla comparativa

Modelo	Categoría Predicha	Confianza
Facebook BART Large MNLI	Sports	44.30%
DeBERTa V3 Zero-Shot	Sports	98.85%
ModernBERT Large Zero-Shot	Sports	94.53%

```python
# ============================================================
# COMPARACIÓN DE RESULTADOS
# ============================================================

import pandas as pd

results = pd.DataFrame({
    "Modelo": [
        "BART Large MNLI",
        "DeBERTa V3",
        "ModernBERT"
    ],
    "Accuracy": [
        bart_accuracy,
        deberta_accuracy,
        modern_accuracy
    ],
    "Precision": [
        bart_precision,
        deberta_precision,
        modern_precision
    ],
    "Recall": [
        bart_recall,
        deberta_recall,
        modern_recall
    ],
    "F1 Score": [
        bart_f1,
        deberta_f1,
        modern_f1
    ]
})

results
```

	Modelo	Accuracy	Precision	Recall	F1 Score
0	BART Large MNLI	0.676667	0.684030	0.676667	0.676277
1	DeBERTa V3	0.840000	0.843719	0.840000	0.836494
2	ModernBERT	0.806667	0.822339	0.806667	0.798790

```python
# ============================================================
# TABLA COMPARATIVA DE LATENCIA
# ============================================================

latency_results = pd.DataFrame({
    "Modelo": [
        "BART Large MNLI",
        "DeBERTa V3",
        "ModernBERT"
    ],
    "Tiempo Total (s)": [
        bart_total_time,
        deberta_total_time,
        modern_total_time
    ],
    "Latencia Promedio (s)": [
        bart_avg_latency,
        deberta_avg_latency,
        modern_avg_latency
    ]
})

latency_results
```

	Modelo	Tiempo Total (s)	Latencia Promedio (s)
0	BART Large MNLI	32.235302	0.107451
1	DeBERTa V3	43.151147	0.143837
2	ModernBERT	47.570681	0.158569

Próximos pasos: Generar código con latency_results | New interactive sheet

Conclusiones parciales.

Se implementaron y evaluaron tres modelos de procesamiento de lenguaje natural disponibles en Hugging Face: Facebook BART Large MNLI, DeBERTa V3 y ModernBERT. Los modelos fueron ejecutados sobre el dataset AG News utilizando una tarea de clasificación de texto basada en cuatro categorías temáticas.

Los resultados obtenidos muestran diferencias en el desempeño de cada modelo. DeBERTa V3 alcanzó los mejores valores de Accuracy (0.84), Precision (0.8437), Recall (0.84) y F1-Score (0.8365), lo que indica una mayor capacidad de clasificación sobre el conjunto de prueba seleccionado.

Respecto a la latencia, Facebook BART Large MNLI presentó el menor tiempo promedio de inferencia (0.107 segundos por registro), seguido por DeBERTa V3 (0.144 segundos) y ModernBERT (0.159 segundos). Esto evidencia que un mejor desempeño predictivo puede estar asociado a un mayor tiempo de procesamiento.

De manera preliminar, DeBERTa V3 ofrece el mejor equilibrio entre precisión y tiempo de respuesta dentro de los modelos evaluados, mientras que BART Large MNLI destaca por su rapidez de inferencia.

2. Documento de análisis comparativo de modelos.

2.1 Descripción del caso de uso

El caso de uso seleccionado consiste en la clasificación automática de noticias mediante técnicas de Procesamiento de Lenguaje Natural (NLP). El objetivo es identificar automáticamente la categoría temática de una noticia a partir de su contenido textual, permitiendo organizar grandes volúmenes de información de manera eficiente y reduciendo la necesidad de clasificación manual.

Para el desarrollo de esta actividad se utilizó el dataset AG News, un conjunto de datos ampliamente utilizado para tareas de clasificación de texto, que contiene noticias clasificadas en cuatro categorías: World, Sports, Business y Sci/Tech. El dataset cuenta con 120,000 registros de entrenamiento y 7,600 registros de prueba, proporcionando una base adecuada para evaluar y comparar el desempeño de distintos modelos de inteligencia artificial.

La evaluación se realizó utilizando modelos preentrenados disponibles en Hugging Face, aprovechando técnicas de clasificación Zero-Shot para determinar la categoría más probable de cada noticia sin necesidad de realizar un entrenamiento adicional.

2.2 Criterios de selección

Para la selección de los modelos se consideraron los siguientes criterios:

- Disponibilidad pública en Hugging Face.
- Compatibilidad con tareas de clasificación de texto.
- Capacidad para realizar inferencia Zero-Shot.
- Popularidad y adopción dentro de la comunidad de NLP.
- Diversidad en arquitectura y complejidad computacional.
- Posibilidad de comparar precisión y tiempos de respuesta.
- Compatibilidad con ejecución mediante GPU en Google Colab.

Con base en estos criterios se seleccionaron los siguientes modelos:

1. Facebook BART Large MNLI.
2. DeBERTa V3 Zero-Shot.

3. ModernBERT Large Zero-Shot.

2.3 Tabla comparativa de modelos

Característica	BART Large MNLI	DeBERTa V3	ModernBERT
Tarea principal	Zero-Shot Classification	Zero-Shot Classification	Zero-Shot Classification
Arquitectura base	BART	DeBERTa V3	ModernBERT
Licencia	MIT	MIT	Apache 2.0
Accuracy	0.6767	0.8400	0.8067
Precision	0.6840	0.8437	0.8223
Recall	0.6767	0.8400	0.8067
F1 Score	0.6763	0.8365	0.7988
Tiempo total (100 registros)	32.24 s	43.15 s	47.57 s
Latencia promedio	0.107 s	0.144 s	0.159 s
Requerimiento computacional	Medio	Medio-Alto	Alto

2.4 Análisis de trade-offs (eficiencia frente a precisión)

Los resultados obtenidos muestran una relación directa entre desempeño predictivo y tiempo de inferencia.

El modelo BART Large MNLI obtuvo el menor tiempo promedio de respuesta, con una latencia de 0.107 segundos por registro. Sin embargo, también presentó los valores más bajos de Accuracy, Precision, Recall y F1-Score.

Por otro lado, DeBERTa V3 alcanzó los mejores resultados en todas las métricas de clasificación evaluadas. Aunque su tiempo de inferencia fue superior al de BART, la diferencia de aproximadamente 0.037 segundos por registro se ve compensada por una mejora significativa en el desempeño predictivo.

ModernBERT obtuvo resultados intermedios en cuanto a precisión, pero presentó la mayor latencia de los tres modelos evaluados.

Desde una perspectiva de eficiencia versus precisión, DeBERTa V3 representa el mejor equilibrio entre ambas variables, ya que logra la mayor calidad de clasificación sin incrementar excesivamente el tiempo de procesamiento.

2.5 Decisiones de ajuste mínimo

Durante el desarrollo de la evaluación se realizaron únicamente ajustes mínimos para mantener condiciones homogéneas entre los modelos:

- Uso del mismo dataset (AG News).
- Uso de las mismas categorías objetivo: World, Sports, Business y Sci/Tech.
- Uso del mismo conjunto de prueba de 100 registros seleccionados aleatoriamente.
- Ejecución en el mismo entorno de hardware utilizando GPU Tesla T4.
- Utilización del pipeline de clasificación Zero-Shot proporcionado por Hugging Face.
- No se realizó entrenamiento adicional ni ajuste de hiperparámetros.

Estas decisiones permitieron que la comparación fuera consistente y que los resultados dependieran principalmente de las capacidades propias de cada modelo.

2.6 Justificación del modelo recomendado

Con base en los resultados obtenidos, el modelo recomendado es **DeBERTa V3 Zero-Shot**.

La recomendación se fundamenta en que fue el modelo con mejor desempeño en todas las métricas de clasificación evaluadas:

- Accuracy: 0.8400
- Precision: 0.8437
- Recall: 0.8400
- F1-Score: 0.8365

Además, aunque su tiempo de inferencia fue superior al de BART, la diferencia observada resulta aceptable considerando la mejora significativa en la calidad de las predicciones. Para

aplicaciones donde la exactitud es un factor crítico, DeBERTa V3 ofrece el mejor equilibrio entre desempeño y eficiencia computacional.

2.7 Umbrales y riesgos

Umbrales propuestos

Para considerar aceptable el desempeño de un modelo en esta tarea se establecieron los siguientes umbrales mínimos:

Métrica	**Umbral mínimo**
Accuracy	≥ 0.75
Precision	≥ 0.75
Recall	≥ 0.75
F1 Score	≥ 0.75
Latencia promedio	≤ 0.20 s

Bajo estos criterios:

- BART Large MNLI no cumple los umbrales de calidad establecidos.
- DeBERTa V3 cumple todos los umbrales.
- ModernBERT cumple todos los umbrales.

Riesgos identificados

- El conjunto de prueba utilizado corresponde únicamente a una muestra de 100 registros.
- El desempeño puede variar al utilizar el conjunto completo de prueba.
- Los modelos Zero-Shot dependen de la formulación de las etiquetas candidatas.
- Los resultados pueden cambiar al utilizar otros dominios o tipos de texto diferentes a noticias.
- La latencia medida depende de la infraestructura utilizada y puede variar en distintos entornos de ejecución.

En consecuencia, se recomienda realizar validaciones adicionales con volúmenes mayores de datos antes de desplegar cualquiera de los modelos en un entorno productivo.

Perfecto. Para el **Punto 3. Dataset y scripts de soporte**, no necesitas inventar nada nuevo; debemos documentar exactamente lo que utilizaste en el notebook.

3. Dataset y scripts de soporte

3.1 Descripción del origen del dataset

Para la ejecución de las pruebas se utilizó el dataset **AG News**, disponible a través de la biblioteca Hugging Face Datasets. Este conjunto de datos fue construido a partir del corpus AG's News y es ampliamente utilizado como benchmark para tareas de clasificación de texto dentro del área de Procesamiento de Lenguaje Natural (NLP). El dataset fue publicado originalmente por Zhang, Zhao y LeCun como parte de sus trabajos de investigación en clasificación automática de textos. [github.com], [tensorflow.org]

El conjunto de datos contiene noticias clasificadas en cuatro categorías temáticas:

- World
- Sports
- Business
- Sci/Tech

La versión utilizada en esta actividad contiene:

- 120,000 registros de entrenamiento.
- 7,600 registros de prueba.

Estas características permiten evaluar de manera objetiva el desempeño de distintos modelos de clasificación de texto utilizando métricas estandarizadas como Accuracy, Precision, Recall y F1-Score.

3.2 Estructura del dataset

La estructura del dataset utilizado se muestra a continuación:

Campo	Tipo	Descripción
text	String	Contenido textual de la noticia.
label	Integer	Categoría asignada a la noticia.

Las etiquetas disponibles son:

Valor	Categoría
0	World
1	Sports
2	Business
3	Sci/Tech

Ejemplo de registro

{

'text': 'Wall St. Bears Claw Back Into the Black...',

'label': 2

}

En este ejemplo, la etiqueta 2 corresponde a la categoría Business.

3.3 Scripts reproducibles

Carga del dataset

El siguiente script fue utilizado para descargar y cargar el dataset desde Hugging Face:

```
from datasets import load_dataset

dataset = load_dataset("fancyzhx/ag_news")

print(dataset)
```

Verificación de la estructura

El siguiente código permitió visualizar un registro de ejemplo:

```
print(dataset["train"][0])
```

Consulta del tamaño del dataset

El siguiente script fue utilizado para validar la cantidad de registros disponibles:

```
print(dataset)

print("Train:", len(dataset["train"]))

print("Test:", len(dataset["test"]))
```

Resultado obtenido:

Train: 120000

Test: 7600

Generación de la muestra de evaluación

Para reducir los tiempos de procesamiento y mantener una evaluación consistente entre los modelos, se seleccionó una muestra aleatoria de 100 registros del conjunto de prueba:

```
sample_test = dataset["test"].shuffle(
        seed=42
).select(range(100))
print("Registros seleccionados:", len(sample_test))
```

Definición de categorías utilizadas

Las etiquetas candidatas empleadas durante la inferencia fueron:

```
candidate_labels = [
        "World",
        "Sports",
        "Business",
        "Sci/Tech"
]
```

Estas categorías corresponden directamente a las clases originales definidas dentro del dataset AG News.

Reproducibilidad del experimento

Para garantizar la reproducibilidad de los resultados se mantuvieron constantes los siguientes elementos:

- Dataset AG News.
- Muestra aleatoria generada con semilla fija (`seed=42`).
- Mismo conjunto de 100 registros para los tres modelos.
- Misma infraestructura de ejecución (Google Colab con GPU Tesla T4).
- Mismas categorías de clasificación.
- Mismo pipeline de inferencia Zero-Shot para todos los modelos evaluados.

De esta manera, las diferencias observadas en precisión y latencia reflejan principalmente el comportamiento propio de cada modelo y no variaciones en los datos o en el entorno de ejecución.

Referencias

Dataset AG News

Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. Advances in Neural Information Processing Systems, 28. https://arxiv.org/abs/1509.01626

BART Large MNLI

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 7871-7880. https://doi.org/10.18653/v1/2020.acl-main.703

DeBERTa V3

He, P., Gao, J., & Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2111.09543

ModernBERT

Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Cooper, N., Adams, G., Howard, J., & Poli, I. (2024). Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. arXiv. https://arxiv.org/abs/2412.13663

Hugging Face

Hugging Face. (s. f.). The AI community building the future. https://huggingface.co/

Transformers Library

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 38-45. https://doi.org/10.18653/v1/2020.emnlp-demos.6